Ciência de Dados para Negócios
Aprendizagem de Máquina - II

Prof. Luis Claudio

ATENÇÃO:

Escrever somente os nomes dos participantes presentes em sala no dia.

Após o final da atividade, o grupo se desfaz (não precisa, necessariamente, manter os mesmos alunos no mesmo grupo para a próxima vez).

Integrantes	**30/09/2026**
1. Ana Elisa Rubinato Cavalheiro	
2. Arthur Nunes dos Santos	
3. Bruno Felipe de Souza Araujo	
4. Caio Cesar Corrá	
5. Nicole Fava Pires	
6.	

Obs.:
- Trabalho presencial. Não coloque nomes de quem não está em sala da aula.
- Mínimo 03 alunos; máximo de 06 alunos.

Situação-Problema

Uma empresa de comércio eletrônico possui milhares de produtos cadastrados em sua plataforma. Durante o cadastro de um novo produto, o vendedor precisa selecionar manualmente sua categoria, como camiseta, calçado, bolsa ou acessório.

A empresa percebeu que erros nessa classificação são frequentes. Produtos cadastrados em categorias incorretas prejudicam pesquisas realizadas pelos clientes, filtros de navegação e algumas análises comerciais.

A equipe de Ciência de Dados recebeu, portanto, a tarefa de estudar uma solução capaz de classificar automaticamente a categoria de um produto a partir de sua imagem.

Para um primeiro experimento controlado, foi preparado um conjunto de treinamento contendo 12.800 imagens distribuídas entre quatro categorias: camiseta, calçado, bolsa e acessório.

Todas as imagens possuem dimensão 32 × 32 pixels e utilizam o padrão RGB.

A equipe decidiu inicialmente construir uma MLP (Multilayer Perceptron), arquitetura já utilizada em outros projetos da empresa.

Como uma MLP recebe seus dados na forma de valores de entrada, cada imagem será transformada em um vetor. Uma imagem RGB de 32 × 32 possui 32 × 32 × 3 = 3.072 valores de entrada.

Arquitetura proposta para a MLP

Parte da rede	Configuração	Ativação
Entrada	3.072 valores	—
1ª camada oculta	32 neurônios	ReLU
2ª camada oculta	8 neurônios	ReLU
Camada de saída	4 neurônios	Softmax

Configuração inicial de treinamento

Hiperparâmetro	Valor
batch_size	128
epochs	25
loss function	Categorical Cross-Entropy
optimizer	Adam
learning rate	0,001

Durante a discussão do projeto, um dos integrantes da equipe chamou atenção para o fato de que, na MLP, cada neurônio da primeira camada oculta estará conectado aos 3.072 valores provenientes da imagem. Outro integrante sugeriu então avaliar uma Rede Neural Convolucional (CNN), arquitetura desenvolvida especialmente para dados com estrutura espacial, como imagens.

Para iniciar essa análise, foi proposta uma primeira camada convolucional:

```
Conv2D(
    filters=12,
    kernel_size=(3,3),
    activation='relu'
)
```

Essa camada receberá diretamente as imagens RGB.

O objetivo da equipe, neste momento, não é determinar qual das duas arquiteturas apresentará melhor desempenho. Antes disso, é necessário compreender como cada uma processa as informações, como seus parâmetros são organizados e como ocorre seu treinamento.

Questões

1. Parâmetros e hiperparâmetros

Considerando a MLP descrita, explique a diferença entre parâmetros e hiperparâmetros.

Resposta: Parâmetros são os valores que a rede aprende sozinha durante o treinamento, ou seja, os pesos e os biases. Eles vão sendo ajustados a cada atualização para diminuir o erro. Hiperparâmetros são escolhas, feitas pelo analista ou cientista antes de treinar. A rede não aprende esses valores, e eles definem como ela é montada e como vai ser treinada.

Em seguida, identifique no projeto:

a) quais elementos serão aprendidos pela rede durante o treinamento;

Aprendidos pela rede: os pesos e os biases de todas as camadas (1ª oculta, 2ª oculta e saída).

b) quais elementos correspondem a hiperparâmetros de arquitetura;

Hiperparâmetros de arquitetura: número de camadas ocultas (2), número de neurônios por camada (32, 8 e 4 na saída) e funções de ativação (ReLU nas ocultas e Softmax na saída).

c) quais correspondem a hiperparâmetros de treinamento.

Hiperparâmetros de treinamento: batch_size = 128, epochs = 25, função de perda Categorical Cross-Entropy, otimizador Adam e learning rate = 0,001.

2. Parâmetros treináveis da MLP

Calcule a quantidade de parâmetros treináveis em cada parte da MLP apresentada. Considere pesos e biases.

Camada de entrada: 0

1ª camada oculta (3.072 → 32): **98.336**

2ª camada oculta (32 → 8): **264**

Camada de saída (8 → 4): **36**

Total de parâmetros treináveis: **98.636**

3. Camada de entrada

Embora a imagem produza 3.072 valores de entrada, a camada de entrada da MLP não acrescenta parâmetros treináveis. Explique:

a) por que essa camada possui zero parâmetros treináveis;

Resposta: A camada de entrada só recebe os 3.072 valores dos pixels e passa eles adiante. Ela não tem pesos, bias nem ativação, então não faz nenhum cálculo e não há nada nela para a rede aprender.

b) por que, mesmo assim, a dimensionalidade da entrada influencia diretamente a quantidade de parâmetros da primeira camada oculta.

Resposta: Na camada densa, cada neurônio da 1ª camada oculta se conecta a todos os valores da entrada, e cada conexão tem um peso. Por isso, quanto mais valores na entrada, mais pesos cada neurônio precisa ter. Com 3.072 entradas, cada um dos 32 neurônios tem 3.072 pesos. Por isso essa camada concentra quase todos os parâmetros da rede (98.336 de 98.636).

4. Batch size e épocas

O conjunto de treinamento possui 12.800 imagens e será utilizado com batch_size = 128 e epochs = 25. Determine:

a) quantos lotes serão processados durante uma época;

Resposta: 12.800 ÷ 128 = **100 lotes por época**.

b) quantas atualizações dos parâmetros serão realizadas durante uma época;

Os pesos e biases são atualizados uma vez ao final de cada lote, então são **100 atualizações por época**.

c) quantas atualizações serão realizadas durante todo o treinamento.

Resposta: 100 atualizações × 25 épocas = **2.500 atualizações no treinamento todo**.

5. Como ocorre o treinamento

Considere que a rede acabou de processar um dos lotes do treinamento. Explique, em sequência, como ocorre o processo de aprendizagem a partir da saída prevista até a atualização dos pesos e biases, destacando o papel da função de perda, do gradiente, da backpropagation, do otimizador e do learning rate.

Resposta: Depois que o lote passa pela rede (forward), a Softmax gera as probabilidades previstas para as 4 categorias. Essas previsões são comparadas com as classes corretas pela função de perda (Categorical Cross-Entropy), que mede o tamanho do erro. Em seguida é calculado o gradiente da perda em relação a cada peso e bias. O gradiente mostra a direção e a intensidade com que cada parâmetro influencia o erro. Esse cálculo é feito pela backpropagation, que leva o erro da saída de volta até a primeira camada, camada por camada, usando a regra da cadeia. Com os gradientes calculados, o otimizador (Adam) decide como atualizar os pesos e biases para diminuir a perda. O learning rate (0,001) define o tamanho desse passo de atualização. Depois disso o próximo lote é processado e o ciclo se repete.

6. Taxa de aprendizagem

A configuração inicial utiliza learning_rate = 0,001. Durante alguns experimentos, dois integrantes sugerem valores muito diferentes: learning_rate = 0,1 e learning_rate = 0,00001. Explique como a taxa de aprendizagem afeta as atualizações realizadas nos pesos e biases, relacionando-a com o gradiente. Se o gradiente indica a direção e a intensidade da influência de cada parâmetro sobre a perda, qual é o papel da taxa de aprendizagem na magnitude da atualização efetivamente realizada?

Resposta: O gradiente mostra para qual lado ajustar cada parâmetro e o quanto ele influencia a perda. A taxa de aprendizagem multiplica o gradiente e define o tamanho real do ajuste: novo peso = peso − learning rate × gradiente. Ou seja, o gradiente dá a direção e o learning rate controla a magnitude do passo.

Com 0,1 os passos ficam grandes demais. A rede pode passar do ponto mínimo, ficar oscilando e até divergir, e a perda não cai de forma estável.

Com 0,00001 os passos ficam muito pequenos. O treinamento fica lento demais e em 25 épocas a rede pode quase não aprender, ou ficar presa em um ponto ruim.

O 0,001 é um meio-termo que costuma funcionar bem com o Adam.

7. Por que considerar uma CNN?

A primeira solução proposta utiliza uma MLP. Posteriormente, a equipe decide estudar uma CNN. Explique por que uma CNN é particularmente adequada para trabalhar com imagens, considerando sua estrutura espacial, o uso de filtros em regiões locais, a reutilização dos mesmos pesos e a geração de mapas de características.

Resposta: A MLP transforma a imagem em um vetor e perde a noção de quais pixels estão perto uns dos outros. A CNN mantém a imagem no formato altura × largura × canais, ou seja, preserva a estrutura espacial. Ela usa filtros pequenos que olham regiões locais da imagem, o que ajuda a detectar padrões como bordas, cantos e texturas. O mesmo filtro (com os mesmos pesos) é reutilizado em toda a imagem. Com isso há muito menos parâmetros, e um padrão é reconhecido em qualquer posição da imagem. Cada filtro gera um mapa de características, que mostra onde aquele padrão aparece. As camadas seguintes combinam esses mapas para reconhecer padrões mais complexos, como o formato de um sapato ou de uma bolsa.

8. Primeira camada convolucional

Considere novamente Conv2D(filters=12, kernel_size=(3,3), activation='relu') recebendo uma imagem RGB. Determine:

a) quantos filtros existem nessa camada;

Resposta: 12 filtros

b) qual é a dimensão completa de cada filtro, considerando os canais da entrada;

Resposta: 3 × 3 × 3 (3×3 de área e 3 canais RGB)

c) quantos mapas de características serão produzidos;

Resposta: 12 mapas de características (um por filtro)

d) quantos pesos existem em cada filtro;

Resposta: 3 × 3 × 3 = 27 pesos por filtro

e) quantos parâmetros treináveis existem em cada filtro, considerando também o bias;

Resposta: 27 + 1 bias = 28 parâmetros por filtro

f) quantos parâmetros treináveis possui toda a camada convolucional.

Resposta: 28 × 12 = **336 parâmetros treináveis**

9. Tamanho da imagem e quantidade de parâmetros

Depois do primeiro experimento, a empresa decide utilizar imagens de maior resolução: 64 × 64 × 3. Mantendo a primeira camada oculta da MLP com 32 neurônios:

a) quantos valores passarão a existir na entrada da MLP;

Resposta: 64 × 64 × 3 = 12.288 valores de entrada

b) quantos parâmetros treináveis passará a possuir somente essa primeira camada oculta;

Resposta: 12.288 × 32 + 32 = 393.216 + 32 = 393.248 parâmetros

c) mantendo filters=12, kernel_size= (3,3) e imagens RGB, quantos parâmetros treináveis continuará possuindo a primeira camada convolucional;

Resposta: Continua com 336 parâmetros, porque o tamanho da imagem não muda nada nesse cálculo.

10. MLP versus CNN

Explique por que aumentar a dimensão da imagem aumenta diretamente a quantidade de parâmetros da primeira camada densa de uma MLP, mas não aumenta a quantidade de parâmetros de uma camada convolucional.

Resposta: Na camada densa da MLP, cada neurônio se liga a todos os valores da entrada, e cada ligação tem seu próprio peso. Se a imagem cresce, crescem junto as ligações e os pesos. De 32×32 para 64×64 a entrada ficou 4 vezes maior, e os parâmetros da primeira camada foram de 98.336 para 393.248. Na CNN, os parâmetros dependem só do tamanho do filtro, do número de canais e do número de filtros (3×3×3×12 + 12). O filtro desliza pela imagem reutilizando sempre os mesmos pesos, então uma imagem maior só aumenta o número de posições por onde ele passa e o tamanho dos mapas de características. A quantidade de parâmetros continua a mesma.

11. Estrutura das duas abordagens

Compare MLP e CNN quanto à forma de representar a imagem, às conexões entre os neurônios e à reutilização de pesos.

Resposta:

- **Representação da imagem:** a MLP "achata" a imagem em um vetor de 3.072 valores e perde a posição e a vizinhança dos pixels. A CNN mantém a imagem no formato 32×32×3 e preserva a estrutura espacial.
- **Conexões:** na MLP cada neurônio se conecta a todos os valores da entrada (totalmente conectada). Na CNN cada saída se conecta só a uma pequena região local (3×3×3).
- **Reutilização de pesos:** na MLP cada conexão tem seu próprio peso, sem nenhum compartilhamento, o que gera muitos parâmetros. Na CNN o mesmo filtro é aplicado em toda a imagem (pesos compartilhados), o que dá bem menos parâmetros e permite reconhecer o mesmo padrão em qualquer lugar da imagem.